

Практическое занятие 25: Настройка админ-панели для управления данными

Цель занятия:

- Познакомиться с админ-панелью Django и ее возможностями.
 - Научиться регистрировать модели в админ-панели.
 - Настроить отображение моделей в админ-панели для удобства использования.
 - Реализовать отображение списка мероприятий на сайте.
-

Задачи занятия

1. Введение в админ-панель Django
 2. Регистрация моделей в админ-панели
 3. Базовая настройка отображения моделей
 4. Реализация отображения списка мероприятий (event_list)
 5. Выполнение практического упражнения. Проверка админ-панели и списка мероприятий
 6. **Выполнение дополнительных заданий. Добавление отзывов, улучшение отображения списка мероприятий
-

Задача 1: Введение в админ-панель Django

1.1. Что такое админ-панель Django?

- **Админ-панель** — встроенный инструмент Django для управления данными приложения.
- Позволяет администраторам просматривать, добавлять, изменять и удалять объекты моделей через удобный веб-интерфейс.
- Предоставляет средства аутентификации и авторизации пользователей.

1.2. Почему важно настроить админ-панель?

- **Удобство управления данными:** Администраторы могут легко управлять данными без необходимости писать код или использовать базу данных напрямую.
- **Экономия времени:** Быстрое создание и редактирование данных во время разработки и тестирования.

- **Расширяемость:** Возможность кастомизировать админ-панель под специфические нужды приложения.
-

Задача 2: Начало регистрации моделей в админ-панели

2.1. Создание суперпользователя

Перед началом работы необходимо создать суперпользователя для доступа к админ-панели.

Шаги:

1. Откройте терминал и выполните команду:

```
python manage.py createsuperuser
```

2. Следуйте инструкциям для ввода имени пользователя, адреса электронной почты (опционально) и пароля.

2.2. Открытие файла `admin.py`

- В приложении `events` откройте файл `admin.py`.

2.3. Импорт моделей

Добавьте следующий код:

```
# events/admin.py

from django.contrib import admin
from .models import Category, Event
```

2.4. Применение миграций (если необходимо)

Если вы недавно добавили новые модели или изменения в модели, выполните миграции:

```
python manage.py makemigrations
python manage.py migrate
```

Задача 3: Базовая настройка отображения моделей

3.1. Настройка модели `Event` в админ-панели

Создайте класс `EventAdmin` :

```
# events/admin.py

class EventAdmin(admin.ModelAdmin):
    list_display = ('title', 'date', 'location', 'category')
```

Зарегистрируйте модель с использованием класса `EventAdmin` :

```
# events/admin.py

admin.site.register(Event, EventAdmin)
```

Пояснение:

- `list_display` определяет, какие поля отображаются в списке объектов модели в админ-панели.

3.2. Настройка модели `Category`

Создайте класс `CategoryAdmin` :

```
# events/admin.py

class CategoryAdmin(admin.ModelAdmin):
    list_display = ('name',)
```

Зарегистрируйте модель `Category` с классом `CategoryAdmin` :

```
# events/admin.py

admin.site.register(Category, CategoryAdmin)
```

3.3. Сохранение и запуск сервера

- Сохраните изменения в `admin.py` .
- Запустите сервер разработки, если он не запущен:

```
python manage.py runserver
```

3.4. Проверка админ-панели

- Перейдите по адресу <http://127.0.0.1:8000/admin/>.
- Введите учетные данные суперпользователя.
- Убедитесь, что модели `Event` и `Category` отображаются и имеют настроенные поля в списке объектов.

Задача 4: Реализация отображения списка мероприятий (`event_list`)

4.1. Создание представления `event_list`

Шаги:

1. Откройте файл `views.py` в приложении `events`.
2. Удалите список `events`
3. Добавьте/измените функцию `event_list`:

```
# events/views.py

from django.shortcuts import render
from .models import Event

# сразу после импорта пропишите новую переменную events
events = Event.objects.all().order_by('date')

def event_list(request):
    print(f"Количество мероприятий: {events.count()}")
    return render(request, 'events/event_list.html', {'events':
events})
```

Пояснение:

- Импортируем модель `Event`.
- Получаем все мероприятия из базы данных, сортируя их по дате.
- Используем функцию `render` для отображения шаблона `event_list.html`, передавая список мероприятий.

4.2. Создание шаблона `event_list.html`

Шаги:

1. Создайте директорию `templates/events/` в вашем приложении `events`, если ее еще нет.
2. Создайте файл `event_list.html` в директории `templates/events/`.
3. Добавьте следующий код в шаблон:

```
<!-- events/templates/events/event_list.html -->

{% extends 'base.html' %}

{% block title %}Список мероприятий{% endblock %}

{% block content %}
    <h1>Список мероприятий</h1>
    <ul>
        {% for event in events %}
            <li>
                <a href="{% url 'events:event_detail'
event.id %}">{{ event.title }}</a> -
                {{ event.date|date:"d.m.Y H:i" }} -
                {{ event.location }}
            </li>
            {% empty %}
            <li>Нет доступных мероприятий.</li>
        {% endfor %}
    </ul>
<!-- Пагинация -->
```

Пояснение:

- Используем цикл `{% for event in events %}` для отображения списка мероприятий.
- Проверяем, есть ли мероприятия, с помощью `{% empty %}`.
- Используем фильтр `date` для форматирования даты.
- Ссылка на детальную страницу мероприятия (`event_detail`).

4.3. Добавление маршрута в `urls.py`

Шаги:

1. Откройте файл `urls.py` в приложении `events`.
2. Добавьте маршрут для `event_list`:

```
# events/urls.py

from django.urls import path
```

```
from . import views

urlpatterns = [
    # ... другие маршруты ...
    path('events/', views.event_list, name='event_list'),
]
```

4.4. Добавление ссылки на список мероприятий

Шаги:

1. Откройте базовый шаблон вашего проекта `base.html`.
2. Добавьте ссылку на страницу списка мероприятий:

```
<!-- templates/base.html -->

<nav>
    <!-- ... другие ссылки ... -->
    <a href="{% url 'events:event_list' %}">Мероприятия</a>
</nav>
```

Пояснение:

- Это позволит пользователям перейти на страницу со списком мероприятий из любой страницы сайта.

4.5. Проверка работы

- Перейдите по адресу <http://127.0.0.1:8000/events/>.
- Убедитесь, что список мероприятий отображается корректно.
- Если у вас нет мероприятий, создайте их через админ-панель или ранее созданные формы.

Задача 7: Практическое упражнение: Проверка админ-панели и списка мероприятий

Задание для студентов:

- Создайте несколько мероприятий через админ-панель.
- Перейдите на страницу списка мероприятий и убедитесь, что они отображаются.

- Проверьте ссылки на детали мероприятия, убедитесь, что они ведут на соответствующие страницы (если у вас есть представление `event_detail`).
- Измените данные мероприятий через админ-панель и убедитесь, что изменения отображаются на сайте.

Цель упражнения:

- Закрепить навыки работы с админ-панелью и отображения данных на сайте.
- Понять связь между данными в базе и их отображением на сайте.

Дополнительное задание

1. Добавление поиска и фильтрации в админ-панели

Цель: Улучшить админ-панель для удобства поиска нужных данных.

Шаги:

1. Добавьте в класс `EventAdmin` поля `search_fields` и `list_filter`:

```
# events/admin.py

class EventAdmin(admin.ModelAdmin):
    list_display = ('title', 'date', 'location', 'category')
    search_fields = ('title', 'description')
    list_filter = ('category', 'date')
```

2. Обновите админ-панель и убедитесь, что поиск и фильтры работают.

2. Улучшение отображения списка мероприятий на сайте

Цель: Сделать список мероприятий на сайте более информативным.

Шаги:

1. Добавьте категорию мероприятия в шаблон `event_list.html`:

```
<!-- events/templates/events/event_list.html -->

<!-- ... ваш существующий код ... -->
{% for event in events %}
    <li>
        <a href="{% url 'event_detail' event.id %}">{{ event.title }}
    </a> -
```

```
        {{ event.date|date:"d.m.Y H:i" }} -  
        {{ event.location }} -  
        Категория: {{ event.category.name }}  
    </li>  
{% empty %}  
    <!-- ... -->  
{% endfor %}
```

2. **Отформатируйте список с помощью таблицы или CSS** для улучшения внешнего вида (опционально).

Заключение

Поздравляем! Вы успешно:

- Настроили админ-панель Django для базового управления данными.
- Научились регистрировать модели и настраивать их отображение в админ-панели.
- Реализовали отображение списка мероприятий на сайте.
- Познакомились с интерфейсом админ-панели и ее возможностями.

Если у вас возникнут дополнительные вопросы или потребуется помощь, не стесняйтесь обращаться.

Успехов в изучении Django!

Вопросы для обсуждения

- **Как админ-панель упрощает управление данными в приложении?**
 - Предоставляет удобный интерфейс для создания, редактирования и удаления данных без необходимости писать код или работать напрямую с базой данных.
 - **Какие преимущества дает реализация списка мероприятий на сайте?**
 - Позволяет пользователям видеть доступные мероприятия, улучшает функциональность сайта, демонстрирует умение работать с представлениями и шаблонами.
-

Полезные ресурсы

- [Документация Django: Админ-панель](#)
- [Введение в админ-панель Django](#)
- [Django Admin Cookbook](#)
- [Django Шаблоны](#)